



Département des Technologies de l'information et de la
communication (TIC)
Filière Informatique et systèmes de communication
Orientation Sécurité informatique

Rapport de recherche

Caméra virtuelle et redirection de flux vidéo

Étudiant

Dylan Oliveira Ramos

Enseignant responsable

Prof. Jean-Marc Bost

Année académique

2025-2026

Yverdon-les-Bains, le 04.03.2026

1 Introduction

Pour pouvoir tromper les sites de vérification d'identité, il faut trouver un moyen de rediriger la vidéo générée vers une caméra détectée comme réelle par ceux-ci. La solution la plus simple est d'utiliser une caméra virtuelle, qui est un périphérique logiciel simulant une caméra physique. Sur Linux, il existe un module noyau appelé `v4l2loopback` qui permet de créer de tels périphériques. En utilisant `FFmpeg`, il est ensuite possible d'envoyer un flux vidéo vers la caméra virtuelle, qui sera alors détectée par les applications comme une caméra réelle.

1.1 Pourquoi Linux ?

Linux est particulièrement adapté pour ce type de tâche car les différentes opérations peuvent être scriptées et automatisées facilement. De plus, Linux permet de reproduire les configurations sur d'autres machines via Docker par exemple, ce qui est idéal pour tester et déployer la solution.

2 Installation

Les commandes qui vont suivre ont été effectuées sur une machine [Ubuntu 24.04](#).

2.1 v4l2loopback

```
1 sudo apt install v4l2loopback-dkms v4l2loopback-utils
```

2.2 FFmpeg

```
1 sudo apt install ffmpeg
```

3 Création d'une caméra virtuelle

La commande ci-dessous crée une caméra virtuelle appelée `VirtualCam` :

```
1 sudo modprobe v4l2loopback devices=1 video_nr=2 card_label="VirtualCam"
exclusive_caps=1
```

- `modprobe v4l2loopback` : charge le module noyau
- `devices=1` : crée 1 périphérique virtuel
- `video_nr=2` : spécifie le numéro du périphérique (erreur s'il existe déjà)
- `card_label="VirtualCam"` : spécifie le nom du périphérique
- `exclusive_caps=1` : rend la caméra compatible avec les applications (simule une caméra réelle)

Il est ensuite possible de lister les caméras disponibles :

```
1 v4l2-ctl --list-devices
```

4 Envoi d'un flux vidéo vers la caméra virtuelle

La commande ci-dessous joue la vidéo `video.mp4` en boucle sur la caméra virtuelle :

```
1 ffmpeg -re -stream_loop -1 -i video.mp4 -f v4l2 -pix_fmt yuv420p /dev/video2
```

- `re` : temps réel (simule une vraie caméra sans envoyer toutes les images en une fois)
- `stream_loop -1` : boucle indéfiniment
- `i video.mp4` : spécifie la vidéo à lancer
- `f v4l2` : spécifie le format de sortie
- `pix_fmt yuv420p` : spécifie le format de la vidéo (utilisé par les vraies caméras)
- `/dev/video2` : spécifie le périphérique de sortie

4.1 Rechargement du module

Avant chaque diffusion de vidéo, il faut recharger le module `v4l2loopback` pour éviter les problèmes de conflits. Cela garantit que la caméra virtuelle est correctement réinitialisée et prête à recevoir le flux vidéo :

```
1 sudo modprobe -r v4l2loopback
2 sudo modprobe v4l2loopback devices=1 video_nr=2 card_label="VirtualCam"
   exclusive_caps=1
```

5 Script d'automatisation

Le script ci-dessous automatise le processus de rechargement du module et de diffusion de la vidéo sur la caméra virtuelle :

```
1 #!/bin/bash
2
3 # Check if video file argument is provided
4 if [ $# -eq 0 ]; then
5     echo "Error: Please provide a video file path"
6     echo "Usage: $0 <video_file>"
7     exit 1
8 fi
9
10 VIDEO_FILE="$1"
11
12 # Check if the video file exists
13 if [ ! -f "$VIDEO_FILE" ]; then
14     echo "Error: Video file '$VIDEO_FILE' not found"
15     exit 1
16 fi
17
18 echo "Removing v4l2loopback module..."
19 sudo modprobe -r v4l2loopback
20
21 echo "Loading v4l2loopback module with virtual camera..."
22 sudo modprobe v4l2loopback devices=1 video_nr=2 card_label="VirtualCam"
23     exclusive_caps=1
24
25 # Small delay to ensure the device is ready
26 sleep 1
27
28 echo "Starting video stream to virtual camera..."
29 echo "Press Ctrl+C to stop streaming"
30 ffmpeg -re -stream_loop -1 -i "$VIDEO_FILE" -f v4l2 -pix_fmt yuv420p /dev/video2
```